

APIs - Application Programming Interfaces

A complete introduction to how software talks to itself.

From class methods to web endpoints, APIs are the bridges that connect it all.

What is an API, really?

An API is a contract: 'If you call me this way, I'll respond that way.'

Think of it as:

- A menu in a restaurant: you don't enter the kitchen, you just ask.
- A remote control: buttons (methods) trigger internal logic you don't see.

APIs live in:

- Classes and Objects
- Libraries
- Files and Databases
- Operating Systems
- Web Services

Python: APIs in Classes

class Calculadora:

```
    def somar(self, a, b):
```

```
        return a + b
```

```
    def subtrair(self, a, b):
```

```
        return a - b
```

somar() and subtrair() are part of the public API.

_validar_numero() would be internal.

Python: APIs in Libraries

```
from datetime import datetime  
print(datetime.now().strftime('%d/%m/%Y'))
```

```
import json  
json.dumps({'nome': 'Joao'})
```

These functions are part of the libraries' public APIs.

JavaScript: APIs in Classes

```
class Usuario {  
  constructor(nome, email) {  
    this.nome = nome;  
    this.email = email;  
  }  
  exibirInfo() { return `${this.nome} - ${this.email}` }  
  validarEmail() { return this.email.includes('@') }  
}
```

Usuario exposes its API through these public methods.

JavaScript: Module APIs

```
// calculadora.js
```

```
export function somar(a, b) { return a + b }
```

```
export function multiplicar(a, b) { return a * b }
```

```
// app.js
```

```
import { somar, multiplicar } from './calculadora.js'
```

The exported functions define the API surface.

APIs Web

When APIs went online, they got URLs.

Example:

GET <https://api.exemplo.com/usuarios?cidade=SP>

Components:

- URL -> resource
- Method -> action
- Headers -> metadata
- Body -> payload

Inside an HTTP Request

POST /api/usuarios HTTP/1.1

Host: exemplo.com

Content-Type: application/json

Authorization: Bearer token123

```
{  
  'nome': 'Joao Silva',  
  'email': 'joao@email.com'  
}
```

Each part tells the server what you want and who you are.

URLs & Query Params

`https://api.exemplo.com/usuarios?idade=25&cidade=SP`

Query parameters let you filter or paginate:

`?nome=joao`

`?page=2&limit=10`

`?sort=nome&order=asc`

Headers

Common Request Headers:

Authorization - your identity

Content-Type - data format

Accept - what you expect back

Response Headers:

Access-Control-Allow-Origin - who can see it

Cache-Control - how long it lives

Frontend <-> Backend

Frontend -> API -> Database

1. Browser sends request
2. API validates and processes
3. Database returns data
4. API sends JSON
5. Frontend renders UI

FastAPI Example

```
from fastapi import FastAPI  
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Usuario(BaseModel):
```

```
    nome: str
```

```
    email: str
```

```
    idade: int
```

```
@app.post('/usuarios')
```

```
def criar_usuario(usuario: Usuario):
```

```
    return {'id': 1, 'nome': usuario.nome}
```

REST: The Old Faithful

REST is a philosophy:

GET /usuarios - list

POST /usuarios - create

GET /usuarios/1 - read

PUT /usuarios/1 - update

DELETE /usuarios/1 - delete

GraphQL: The Rebel Child

```
query {  
  usuario(id: 123) {  
    nome  
    pedidos { id total }  
  }  
}
```

One request, exact data, no waste.

No over-fetching or under-fetching.

REST vs GraphQL

Feature | REST | GraphQL

-----|-----|-----

Multiple endpoints | Yes | One

Over-fetching | Common | No

Flexibility | Limited | High

Caching | Simple | Complex

Adoption | High | Growing

APIs Beyond Code

APIs mold the digital world:

- Weather apps
- Logins via OAuth
- Spotify playlists
- IoT devices

APIs are the bloodstream of the internet.

Recap: Why APIs Matter

Reuse

Integration

Scalability

Automation

Modularity

Every layer of software is stitched together by APIs.

Thank You

Everything that connects software is an API.

Code smart. Design APIs that make sense.

Because great APIs make great systems.